

Heuristic v5: Utility-Density Aware Mapping Refinement

Post-SPS Balancing of Optional-Segment Utility Potential

Technical Report | USRT Scheduling System

1. Motivation and Problem

1.1 The USRT Problem

The Utility-Aware Scheduling with Real-Time constraints (USRT) problem maps periodic real-time tasks to processors. Each task T_i has:

- A mandatory segment (execution time e_m) that must complete before the deadline.
- N_{seg} optional segments with incremental execution times. Running them yields utility.
- A utility rate u_i : reward per unit of optional execution time.

The objective is to maximise total utility subject to EDF timing feasibility and a global energy budget B .

1.2 What SPS Does and Misses

Phase 1 uses SPS (Sum Partial Solutions) to map jobs to processors. Its load metric is mandatory utilisation: $l_i = e_m / p_i$. SPS minimises the gap between the most- and least-loaded processor, producing a mapping where every processor carries approximately equal mandatory load. This is correct for EDF feasibility (Liu & Layland: EDF schedules a task set on one processor if and only if total utilisation ≤ 1.0).

THE SPS INADEQUACY: SPS knows nothing about optional segments or utility. It can accidentally map all high-utility tasks onto one processor. Even if the energy budget allows their optional segments to run, the timing constraint on that congested processor prevents it. The other processor has free time but nothing valuable to fill it with.

1.3 Heuristic v5 - the Fix

Heuristic v5 inserts a new Phase 1b (Refine Mapping) immediately after Phase 1 (SPS). It adjusts which jobs sit on which processor to spread utility potential evenly, without breaking the feasible mandatory mapping that SPS produced.

Two algorithms are provided:

- v5a (Step 2a): single linear scan over all doublets.
- v5b (Step 2b): iterative one-swap-one-scan, always targeting the globally most overloaded processor.

Phases 2-6 (left-shift diagnostic, energy check, frequency scaling, greedy optional segments, pairwise swap) are identical to Heuristic v4.

2. Key Definitions

2.1 Cumulative Execution Time

Let $c[i][k] = e_{\{i,0\}} + e_{\{i,1\}} + \dots + e_{\{i,k\}}$ (at f_{max} , $k = 0..N_{seg}$).

So $c[i][0] = e_m$ (mandatory only) and $c[i][N_{seg}] = \text{full execution if all segments run}$.

2.2 Utility Density of a Task

$$ud(i) = u_i \times (c[i][N_{seg}] - c[i][0]) / p_i$$

Plain English: if this task always ran all its optional segments, how much utility would it generate per unit of clock time?

u_i reward per unit of optional execution time
 $c[i][N_{seg}] - c[i][0]$ total optional execution time available per job
 $1 / p_i$ how many times per unit clock time this task recurs

Every job of task T_i has the same $ud(i)$. A task with $ud=5.0$ is 5x more valuable per clock-second than one with $ud=1.0$.

2.3 Processor Utility Density

$$UD(x) = \text{sum of } ud(i) \text{ for all jobs } (i,j) \text{ mapped to processor } x$$

The total utility potential concentrated on processor x. The refinement goal is to make UD(x) approximately equal across all processors.

2.4 Mandatory Utilisation

$$\mu(i) = e_m(i) / p_i$$

$$MU(x) = \text{sum of } \mu(i) \text{ over all jobs on } x$$

SPS balances MU(x). The refinement must preserve this balance (see Condition 2 in Section 4).

3. Doublets

A doublet is any pair of two jobs drawn from the full job set. All $C(N_{\text{jobs}}, 2)$ pairs are formed and sorted by ASCENDING $|ud(a) - ud(b)|$. Pairs with similar utility density come first; pairs with very different utility density come last.

WHY THIS ORDER? Jobs with similar ud are the most interchangeable - swapping them between processors barely shifts the UD distribution. Jobs with very different ud are the big moves. Sorted ascending, the list moves from safe, small adjustments up to large disruptive ones. Both 2a and 2b scan this list from the front.

Doublets are built once before any swaps and reused throughout. Each pair is visited at most a bounded number of times.

4. Swap Acceptance Conditions

Any proposed swap of job a (on Pa) with job b (on Pb) must pass ALL THREE of the following conditions before being committed. Conditions 1 and 2 are O(1) checks. Condition 3 (DBF) is only reached if both pass, so the expensive window check is rarely invoked.

Condition 1 -- UD Imbalance Decreases

Simulate the swap on temporary values (real data is NOT modified):

$$UD'(Pa) = UD(Pa) - ud(a) + ud(b)$$

$$UD'(Pb) = UD(Pb) - ud(b) + ud(a)$$

For Step 2a (pair-level check):

$$|UD'(Pa) - UD'(Pb)| < |UD(Pa) - UD(Pb)| - \epsilon$$

The UD imbalance between these two specific processors must strictly decrease.

For Step 2b (global max-UD check):

$$\max(UD'(P1), UD'(Pb)) < UD(P1) - \epsilon$$

where P1 is the globally highest-UD processor. The new maximum across both affected processors must be strictly less than P1's current UD. This also prevents cycling: every accepted swap strictly reduces the global max.

Condition 2 -- Mandatory Utilisation Balance Does Not Worsen

WHY THIS GUARD IS NEEDED. Without it, a swap that improves UD balance might move a high- e_m job onto an already-loaded processor. That processor then has little timing slack left for optional segments, even though UD is balanced on paper. Phase 5 finds fewer optional segments can fit, and total utility drops. This was the root cause of the low-utility failure before this guard was added.

$$\text{delta_cur} = |MU(Pa) - MU(Pb)|$$

$$\text{delta_new} = |(MU(Pa) - \mu(a) + \mu(b)) - (MU(Pb) - \mu(b) + \mu(a))|$$

$$\text{Accept only if: } \text{delta_new} \leq \text{delta_cur} + \epsilon$$

Condition 3 -- DBF Timing Feasibility

Build hypothetical job lists for both processors after the swap (temporary copies only). Run a mandatory-only DBF check at f_{max} on both:

For every window $[t1, t2]$ on processor x:

$$\text{sum of } e_m \text{ for all jobs with release } \geq t1 \text{ and deadline } \leq t2 \leq (t2 - t1)$$

If either processor violates this, the swap is rejected. This is the hard correctness guarantee: no committed swap ever makes the mandatory schedule infeasible under EDF. Only if all three conditions pass does `_do_swap()` get called to permanently update the six tracked data structures.

5. Algorithm Step 2a - Linear Doublet Scan (v5a)

Algorithm: Refine Mapping - Step 2a

```

Compute ud(i), mu(i) for all tasks
Initialise UD(x), MU(x) for all processors from current mapping
Build doublets = all job pairs, sorted by ascending |ud(a)-ud(b)|

FOR EACH (a, b) IN doublets:           # visits every pair exactly once
    Pa = mapping[a]; Pb = mapping[b]
    IF Pa == Pb: CONTINUE               # same processor, skip

    Compute UD'(Pa), UD'(Pb)           # Condition 1 check
    IF |UD'(Pa)-UD'(Pb)| >= |UD(Pa)-UD(Pb)| - eps: CONTINUE

    Compute delta_new                   # Condition 2 check
    IF delta_new > delta_cur + eps: CONTINUE

    IF SwapFeasible(a, b, Pa, Pb):     # Condition 3 (DBF check)
        DoSwap(a, b, Pa, Pb)          # commit swap
        # NO break -- continue to next pair in list

END FOR

```

Key property of 2a: the loop NEVER breaks early. Every doublet pair is visited exactly once. When a swap is committed, UD and MU are updated immediately so subsequent pairs see the current state. This is a DISTRIBUTED approach: it fixes many local imbalances in a single left-to-right pass without ever re-identifying the globally worst processor.

6. Algorithm Step 2b - One-Swap-One-Scan (v5b)

Algorithm: Refine Mapping - Step 2b

```

Compute ud(i), mu(i); initialise UD(x), MU(x)
Build doublets as in Step 2a

OUTER LOOP:
    P1 = processor with max UD(x)      # re-identified every iteration
    P2 = processor with min UD(x)
    IF UD(P1) - UD(P2) < eps: BREAK    # already balanced

    swapped = False
    FOR EACH (a, b) IN doublets:      # inner scan from the beginning
        hi = job with higher ud in pair
        lo = the other job
        IF mapping[hi] != P1: CONTINUE # hi-ud job must be on P1
        Pb = mapping[lo]
        IF Pb == P1: CONTINUE

        IF max(UD'(P1), UD'(Pb)) >= UD(P1) - eps: CONTINUE # Condition 1
        IF delta_new > delta_cur + eps: CONTINUE           # Condition 2
        IF SwapFeasible(hi, lo, P1, Pb):                  # Condition 3
            DoSwap(hi, lo, P1, Pb)
            swapped = True
            BREAK                                           # exit inner, go back to outer
    END FOR

    IF NOT swapped: BREAK                # no valid swap found: done
END LOOP

```

Key properties of 2b:

BREAK after first valid swap: the inner loop is abandoned immediately. The outer loop goes back to the top and re-identifies P1 from the updated UD values. The next inner scan restarts from the beginning of the doublet list.

Always targets P1: the swap always moves a job off the globally most overloaded processor. This is the GREEDY approach.

Termination guarantee: Condition 1 for 2b requires $\max(UD'(P1), UD'(Pb)) < UD(P1)$. Every accepted swap strictly reduces the global maximum UD. Since $UD \geq 0$, the outer loop must terminate.

7. DoSwap - Data Structures Updated

When a swap of job a (on Pa) with job b (on Pb) is committed, six quantities are updated atomically:

Data Structure	Update after swap $a(Pa) \leftrightarrow b(Pb)$
mapping[a]	Pa -> Pb
mapping[b]	Pb -> Pa
proc_jobs[Pa]	remove a, add b
proc_jobs[Pb]	remove b, add a
UD(Pa)	$UD(Pa) - ud(a) + ud(b)$
UD(Pb)	$UD(Pb) - ud(b) + ud(a)$
MU(Pa)	$MU(Pa) - mu(a) + mu(b)$
MU(Pb)	$MU(Pb) - mu(b) + mu(a)$

UD and MU are running totals updated incrementally after each swap. Conditions 1 and 2 are therefore $O(1)$ per candidate pair. Condition 3 (DBF) is only computed after 1 and 2 pass.

8. Comparison: Step 2a vs Step 2b

Property	Step 2a (v5a)	Step 2b (v5b)
Structure	Single linear pass over all doublets	Outer loop + inner scan, repeated
Target processor	Whichever processors the current pair happens to	Always the globally highest-UD processor P1
After a swap	Continue to next pair (no break)	Break inner; re-identify P1; restart scan
Doublet scans	Exactly one	One per accepted swap
UD condition	Pair-level $ UD(Pa) - UD(Pb) $ decreases	Global max UD strictly decreases
Approach	Distributed: fixes many local imbalances	Greedy: always attacks the single biggest problem
Worst-case cost	$O(N^2 * W^2)$ once	$O(k * N^2 * W^2)$, k = accepted swaps

9. Why SPS Uses Utilisation and Why UD is the Right Metric

9.1 Why SPS uses e_m/p_i and not raw e_m

Raw execution time e_m alone is meaningless for scheduling balance. A job with $e_m=10$ and $p=10$ occupies the processor 100% of the time; one with $e_m=10$ and $p=100$ occupies only 10%. Utilisation e_m/p_i normalises for period and measures the fraction of processor time consumed per unit clock time. Balancing utilisation gives equal mandatory load, which is exactly the condition for EDF feasibility. All SPS feasibility checks and gap computations work in utilisation units, not raw execution time.

9.2 Comparing refinement metric candidates

Metric	Reward?	Amount?	Key Problem
e_m/p_i (mand. util.)	No	No	SPS already balances this. Refinement is a no-op.
u_i only	Yes (rate)	No	Ignores how much optional work exists and how often the task
e_{opt}/p_i (opt. exec.)	No	Yes	Treats all optional work equally. Misleads when u_i varies.
$u_i \times e_{opt}/p_i$ (UD)	Yes	Yes	Best available proxy. Overestimates achievable; needs mutil guard.

UD = 'if all optional segments ran, how much utility per unit clock time?' It correctly weights tasks that are frequent (small

p_i), valuable (large u_i), and have large optional work (large e_{opt}).

Known limitation: UD uses the theoretical maximum (all segments). The actual achievable utility per processor cannot be computed without running Phase 5 first (chicken-and-egg problem). UD is therefore the best available proxy computable before scheduling begins.

10. Full Pipeline - Heuristic v5

Phase	Name	What it does
1	Quantum SPS	Maps jobs balancing e_m/p_i . Knows nothing about utility.
1b	Refine Mapping (NEW)	Rebalances $UD(x)$ via doublet swaps. Preserves $MU(x)$ and DBF.
2	Left Shift	Diagnostic: confirms timing slack at f_{max} .
3	Energy Slack	Checks if mandatory segments at f_{max} fit within budget B.
4	Freq. Scaling	Attempts to lower frequency to save energy.
5	Greedy Optional	Adds optional segments by utility density; checks DBF + energy.
6	Pairwise Swap	Swaps low-value optional segs for high-value; fill pass after each.

WHY PHASE 1b HELPS PHASE 5. Phase 5 can only schedule optional segments where timing slack exists. Slack on processor x is proportional to $1 - MU(x)$. If Phase 1b has spread high-UD jobs across processors while keeping $MU(x)$ balanced, then (a) every processor has similar timing slack available, and (b) every processor has high-utility jobs ready to fill that slack. Phase 5 can exploit capacity on all processors instead of being blocked on the congested one and idle on the empty one.

11. Concrete Worked Example

2 processors, 4 tasks. All tasks have period $p = 1$, hyper-period $h = 10$.

Task	e_m	p	u_i	e_{opt}	$mu(i)=e_m/p$	$ud(i)=u*e_{opt}/p$
T_A	0.40	1	8	0.50	0.40	4.00
T_B	0.30	1	7	0.40	0.30	2.80
T_C	0.10	1	1	0.30	0.10	0.30
T_D	0.10	1	1	0.20	0.10	0.20

State after SPS (worst-case imbalanced outcome):

$P0 = \{T_A, T_B\}$ $MU(P0) = 0.70$ $UD(P0) = 6.80$

$P1 = \{T_C, T_D\}$ $MU(P1) = 0.20$ $UD(P1) = 0.50$

MU is imbalanced; SPS had limited choice in this small example. More critically, UD is catastrophically skewed: all valuable optional work is on $P0$, which has only 30% of its time free for optional segments.

Doublet pairs sorted by $|ud(a) - ud(b)|$:

Pair	$ ud_a - ud_b $	Same processor?	Action in 2b
(T_C, T_D)	0.10	Yes (P1)	Skip (same proc)
(T_A, T_B)	1.20	Yes (P0)	Skip (same proc)
(T_B, T_C)	2.50	No	Candidate (T_B is hi_job on $P0=P1$)
(T_B, T_D)	2.60	No	Candidate
(T_A, T_C)	3.70	No	Candidate
(T_A, T_D)	3.80	No	Candidate

Step 2b trace:

Outer: $P1 = P0$ ($UD=6.80$). Scan doublets for first pair with hi_job on $P0$.

Pair (T_B, T_C): T_B is hi_job ($ud=2.8$, on $P0$). T_C is lo_job ($ud=0.3$, on $P1$).

Condition 1:

$$UD'(P0) = 6.80 - 2.80 + 0.30 = 4.30$$

$UD'(P1) = 0.50 - 0.30 + 2.80 = 3.00$
 $\max(4.30, 3.00) = 4.30 < 6.80 \rightarrow \text{PASS}$

Condition 2:

$\mu(T_B)=0.30, \mu(T_C)=0.10$
 $MU'(P0) = 0.70 - 0.30 + 0.10 = 0.50$
 $MU'(P1) = 0.20 - 0.10 + 0.30 = 0.40$
 $\delta_{\text{new}} = |0.50 - 0.40| = 0.10 < \delta_{\text{cur}} = |0.70 - 0.20| = 0.50 \rightarrow \text{PASS}$

Condition 3 (DBF): both processors feasible at mandatory load. $\rightarrow \text{PASS}$

SWAP COMMITTED: T_B goes to P1, T_C goes to P0.

New state:

$P0 = \{T_A, T_C\} \quad MU(P0) = 0.50 \quad UD(P0) = 4.30$
 $P1 = \{T_B, T_D\} \quad MU(P1) = 0.40 \quad UD(P1) = 3.00$

Re-identify $P1 = P0$ ($UD=4.30$). Next candidate: (T_A, T_D), T_A on P0.

$UD'(P0) = 4.30 - 4.0 + 0.2 = 0.50 \quad UD'(P1) = 3.0 - 0.2 + 4.0 = 6.80$
 $\max(0.50, 6.80) = 6.80 \geq 4.30 \rightarrow \text{FAIL Condition 1. Rejected.}$

No further valid swaps found. Algorithm terminates.

Result: UD spread reduced from 6.30 to 1.30. MU spread reduced from 0.50 to 0.10. Phase 5 now sees balanced mandatory load AND balanced utility potential on both processors.

12. Summary

- SPS balances mandatory utilisation e_m/p_i (utilisation units, not raw execution time) because utilisation directly measures processor time fraction.
- Phase 1b (Refine Mapping) fixes SPS's blindness to utility by rebalancing $ud(i) = u_i * (c[i][N_seg] - c[i][0]) / p_i$ across processors.
- Doublets (all job pairs sorted by ascending $|ud_a - ud_b|$) are the candidates for swapping. Each is considered a bounded number of times.
- Every swap must pass three conditions: UD improves (Condition 1), mandatory utilisation balance does not worsen (Condition 2), DBF remains feasible (Condition 3).
- v5a makes a single linear pass over all doublets (distributed, one scan). v5b iterates and always targets the globally most overloaded processor (greedy).
- The mandatory utilisation guard (Condition 2) was essential: without it, UD-focused swaps disrupted timing slack and reduced the utility Phase 5 could achieve.
- UD is the best cheap proxy for utility potential before scheduling, correctly combining reward rate u_i , optional volume e_{opt} , and task frequency $1/p_i$ in one number.